

Tells — Rebuild & Design Guide

A complete account of **what** this project is, **why** it was built the way it was, and **how** to rebuild it from scratch — including the tech stack, architecture, every scoring/anti-cheat constant, and the design decisions behind each choice.

1. What it is

Tells is a competitive, browser-based **AI-security awareness game** for a broad workplace audience (developers, ERP consultants, HR, admins). It's four short, timed rounds — each an ordinary work task that quietly teaches one AI/security lesson — plus a login gate, anti-cheat, hints, a leaderboard, and a per-player results report.

The four rounds and the competency each measures:

#	Round	Competency it teaches
1	The assistant's inbox	Spotting AI hijacking (prompt injection)
2	The data gate	Protecting the data you give to AI
3	Face value	Detecting AI fakes & manipulated content
4	The review board	AI governance & fairness

2. Why it was built this way — the brief

The guiding constraints, established up front, shaped everything:

- **Audience: all employees, not a SOC team.** Every lesson must land on a non-technical person playing once, while not boring the technical person in the room.
- **Format: a competitive, timed event.** Time pressure is realistic (real phishing exploits urgency) and a leaderboard drives engagement that compliance training lacks.
- **It must not feel like mandatory training.** The learning is smuggled *inside* a game loop — you're trying to win, and getting better at the real skill is the side effect. We deliberately avoided lecture-style "now demonstrate the takeaway" steps.
- **"Guessing must be expensive."** Under a timer, free guessing teaches nothing. Every round makes a confident wrong answer cost more than taking a few seconds to reason, and both error types cost (missing a threat *and* over-flagging a safe thing), so the winning strategy is *look* → *recognise* → *commit*, not spam-click.
- **Feel comes from reactivity, not graphics.** A corporate audience rejects cartoons but accepts a slick "tactical dashboard": dark UI, sharp accents, instant feedback, a draining timer. That's why the aesthetic is a reactive console, not illustrations.

These are the north stars. When a later decision is unclear, it was resolved by asking: *does a non-technical player learn the lesson in ~90s? does guessing lose? is it fair to rank people on?*

3. Tech stack & why

Layer	Choice	Why
Framework	Next.js 14.2.5 (App Router)	File-based routing = one folder per round; static export-friendly; zero-config TS/Tailwind.
Language	TypeScript 5.5	Round data is structured; types catch content/scoring mistakes.
UI	React 18 (Server + Client Components)	Providers/interactivity are client; the shell is server-rendered.
Styling	Tailwind CSS 3.4 + a small <code>globals.css</code>	Fast, consistent design tokens; theme colors centralised in <code>tailwind.config.ts</code> .
State	React Context + <code>localStorage</code>	No backend needed for a demo; progress/score/leaderboard persist per device.
Effects	HTML <code><canvas></code> (Matrix rain), CSS selection trick (hidden text)	Self-contained, no assets, offline.
Build/deps	<code>postcss</code> , <code>autoprefixer</code> only	No runtime dependencies beyond React/Next — trivial to host, audit, and run offline.

Deliberately **no**: state library (Redux/Zustand), component kit (MUI/shadcn), animation lib (Framer), or backend. The whole thing is static + `localStorage`, which is the right weight for a controlled event and keeps the "move this server-side for production" line crisp (see §12).

4. Architecture

```
Browser (all client-side)
├─ RootLayout (app/layout.tsx, server)
│   ├─ <MatrixRain/>      canvas, fixed, z-[-10]      (background)
│   ├─ <Logo/>            fixed top-left              (branding)
│   ├─ <GameProvider>     progress/score (localStorage: chimera_game_v1)
│   │   └─ <AuthProvider> session + anti-cheat (localStorage: tells_user_v1, tells_anticheat)
│   │       ├─ <SessionGuard/> tab-switch monitor + locked/DQ route guard
│   │       ├─ <AuthGate>    renders <Login/> until authenticated, else the page
│   │       │   └─ children = the routed page (/ , /round-N, /debrief)
│   │       └─ <UserMenu/>    top-right initials → dropdown (rules / log out); auto-opens r
│   └─ <Fullscreen/>       bottom-right toggle + auto-request on first gesture
```

Data flow: each round page owns its own play state (React state + refs), computes a `RoundResult`, and writes it via `GameProvider.recordResult(slug, result)`. The mission map, the results page, and the leaderboard all read from `GameProvider` + `AuthProvider`.

Why refs alongside state: a round finalises (records its score) immediately after the last action, and React state updates are async — reading state in `finalize()` gives a *stale* value. So each round mirrors its running score/answers into `useRef`s and records from the refs. (This was the root cause of a real bug where Round 4's "cleared" card showed fewer points than the player earned.)

5. Directory map

```
tells-app/
├─ app/
│   ├─ layout.tsx          root shell: providers + global overlays
│   ├─ page.tsx            mission map (round cards, sequential-lock UI, effective score)
│   ├─ globals.css         base bg, ::selection reveal, hidden-ink, scrollbars
│   ├─ icon.svg            favicon (white flag) – Next auto-detects
│   ├─ debrief/page.tsx    results: Leaderboard tab + "How you did" tab
│   └─ round-1..4/page.tsx the four rounds (self-contained)
├─ components/
│   ├─ HudBar.tsx          timer bar + score + extra stat + hint slot
│   ├─ IntroOverlay.tsx    briefing: objective + numbered how-to
│   ├─ DebriefOverlay.tsx  per-round debrief: stats, breakdown, lesson, live-system consequences
│   ├─ ObjectiveBar.tsx    persistent one-line objective under the HUD
│   ├─ Hints.tsx           hint popover (free theme hint, then costed)
│   ├─ Login.tsx           login form
│   ├─ AuthGate.tsx        gate: login vs app
│   ├─ UserMenu.tsx        top-right initials + dropdown + auto rules-on-login
│   ├─ RulesModal.tsx      scoring / speed bonus / anti-cheat rules
│   ├─ SessionGuard.tsx    tab-switch anti-cheat + locked/DQ route redirect
│   ├─ MatrixRain.tsx      canvas digital-rain background
│   ├─ Fullscreen.tsx      fullscreen toggle + first-gesture auto
│   ├─ Logo.tsx            top-left company logo
│   └─ Icon.tsx            inline SVG icon set (one component, path table)
├─ lib/
│   ├─ config.ts           GAME_TITLE, tagline, timeBonus(), HINT_COST
│   ├─ rounds.ts           ROUNDS registry (title/blurb/competency/takeaway/focus) + SEQUENTIAL
│   ├─ gameState.tsx       GameProvider (progress/score persistence)
│   ├─ auth.tsx            AuthProvider (credentials, session, anti-cheat)
│   ├─ leaderboard.ts      seed + per-device persistence + ranking
│   ├─ useCountdown.ts     1s timer with a ref for accurate finalize reads
│   └─ types.ts            RoundResult and round-1 legacy types
└─ public/assets/company-logo.png
```

6. Cross-cutting systems

Game state (`lib/gameState.tsx`)

`GameProvider` holds `results: Record<slug, RoundResult>` in `localStorage` (`chimera_game_v1`).
`RoundResult` = { `score`, `correct`, `total`, `accuracy`, `breaches`, `flagsCaptured`, `flagsTotal` }.
`totalScore` = sum of `results[].score`. API: `recordResult`, `isComplete`, `reset`.

Timer (lib/useCountdown.ts)

A 1-second countdown returning `{ timeLeft, timeLeftRef, reset }`. Only the round page consumes it; the timer **bounds** a round (fires `finalize()` at 0) and drives the HUD bar. `timeLeftRef` exists so `finalize()` reads an accurate remaining value for the time bonus regardless of React's async state.

Scoring model

Points are **fixed per correct decision** with **symmetric penalties**, plus a **speed bonus** and **hint costs**. See §9 for the full table. Principles: getting it right scores; the *dangerous* mistake and the *over-cautious* mistake both cost; speed is a reward, not the main driver.

Time bonus (lib/config.ts → timeBonus)

On finishing a round, leftover seconds convert to points: **+2/sec, capped at +300**. Added to `scoreRef` in each round's `finalize()`, recorded into the result, and shown as a "time bonus" stat in the debrief. Time-out $\Rightarrow$ 0 bonus. This makes "fast **and** right" win without letting speed outweigh accuracy.

Hints (components/Hints.tsx)

A HUD button (right of the score). **First hint is free** and explains the round's *theme* (a definition); **each further hint costs 75 pts** and gets more specific about clearing that round. Costs deduct from the live round score via an `onSpend` callback the round supplies (subtract from `scoreRef / bump(-c)`). Hint text lives per-round as a `HINTS: string[]` (index 0 = free theme).

Hidden-text discovery (realistic, no button)

Hidden instructions render as **real, selectable text coloured to match the panel** (`.hidden-ink`) — invisible at rest. A global `::selection` rule forces a readable colour, so **drag-highlighting (or Ctrl/Cmd+A) reveals** white-on-white text, exactly as you'd uncover it in a real email/PDF. There is intentionally no "inspect" button — the discovery *is* the transferable habit.

Auth (lib/auth.tsx)

`AuthProvider` validates against a hard-coded `CREDENTIALS` map (demo: users `alex`, `sam`, `jordan`, `priya`, `admin`; password `tells2026`), stores the session in `localStorage` (`tells_user_v1`), derives initials, and exposes a `justLoggedIn` flag so the rules modal auto-opens on an actual login (not on reload/restore). It also owns the **anti-cheat** state (below), persisted to `tells_anticheat_v1`.

Anti-cheat (components/SessionGuard.tsx + auth.tsx)

While on a `/round-*` route, `visibilitychange  $\rightarrow$  hidden` (tab switch / minimise) calls `registerTabSwitch()`: **-150 pts each**, and at **3 switches $\rightarrow$ disqualified**. The penalty is a global deduction (`effective = totalScore - penalty`) shown on the home screen, user menu, and leaderboard; DQ locks rounds and shows a "disqualified" marker. `SessionGuard` also redirects away from any round that's locked or (when DQ'd) all rounds.

Sequential unlock (`lib/rounds.ts` → `SEQUENTIAL_LOCK`)

When `true`, round n unlocks only once round $n-1$ is cleared. **Currently `false`** (all rounds playable) per request; flip to `true` to enforce order at a live event. Both the mission map and `SessionGuard` respect the flag.

Leaderboard (`lib/leaderboard.ts`)

A few seeded competitors + real logins persisted per device (`tells_leaderboard_v1`), merged with the current user's live effective score and DQ status, sorted (DQ to the bottom). Rendered in the Leaderboard tab of the results page.

Briefings & debriefs

- **IntroOverlay** — every round opens with a highlighted *objective* and a numbered *how-to-play* list.
- **DebriefOverlay** — after each round: a stats row, a per-item breakdown, a "**what this round was really about**" lesson, and — only if you erred — an "**if this were a live system**" block spelling out the *real-world consequence of your specific mistakes*.
- **Results page** — Leaderboard + a "How you did" competency report (strengths, focus areas, per-round bands, and a takeaway checklist).

Atmosphere

- **MatrixRain** — a transparent `<canvas>` fixed behind everything, columns of `0/1` with a bright head and fading tail (~16fps, `prefers-reduced-motion` aware). Modal backdrops are translucent (`/70`) so the rain shows through, frosted.
- **Fullscreen** — native fullscreen can't be forced on load (browsers require a gesture), so it's requested on the *first click* and via an always-available toggle.
- **Logo** — top-left on every screen from `public/assets/company-logo.png` ; the HudBar reserves left padding so they never overlap.

7. The four rounds in detail

Round 1 — The assistant's inbox (prompt injection)

- **Concept:** an AI assistant auto-acts on incoming email/docs/tickets/web pages; attackers hide instructions in that content to hijack it.
- **Mechanic:** 6 items (3 genuine / 3 attacks), each shown as *incoming content* → *the assistant's proposed action* → *your decision* (*Approve / Block*). Ordered easy→hard: the genuine messages get more alarming-looking (raising over-block temptation) while attacks get better camouflaged — a labelled instruction buried in a long email → hidden white-on-white text (reveal by selecting) → an unlabelled instruction woven into a retrieved help page (indirect injection).
- **Scoring:** +200 correct · **-250** approving a hijack (it runs) · **-100** blocking a genuine message · timer 240s.

- **Lesson:** an AI can't reliably separate content from commands; check what it's about to *do*, and treat retrieved content as untrusted too.

Round 2 — The data gate (protecting data given to AI)

- **Concept:** three different AI systems, three kinds of data, three different correct control sets. The point: guardrails are **context-dependent** — no universal checklist.
- **Mechanic:** for each system you see its system prompt, a concrete sample record (with sensitive fields highlighted) and a "**what could go wrong here**" risk list; you **arm 3 of 6 controls** (one per risk) and open the gate. Every control has a plain-language description shown *before* you choose, so it's reasoning, not jargon-recall.
 - Systems: **support-ticket summariser** (redact / no-log / no-retention), **internal code assistant** (secret-scan / no-training tenancy / access-scoped retrieval), **CV screening** (de-bias / audit trail / human-in-the-loop).
 - Centrepiece: the **same control flips** — "keep a log" is a *leak* for the support tickets but a *requirement* for the auditable hiring decision.
- **Scoring:** +250 per system secured (+150 first-try) · **−80** per wrong gate-open · timer 240s. Wrong opens increment a "leaks" counter (feeds the report's "issues missed").
- **Lesson:** a written instruction isn't a control; ask what the model *receives, keeps, or is accountable for*.

Round 3 — Face value (real / fake / tampered)

- **Concept:** three escalating tests under one shared clock; later tests worth more.
- **Mechanic:**
 - **Test 1 — Human or AI?** classify samples Human / AI / **can't tell** (honest uncertainty scores highest).
 - **Test 2 — Real or fabricated applicant?** a rendered résumé with a photorealistic (StyleGAN) headshot, portfolio and reference; flag the fakes and, in a **reason modal**, say *why* (everyday tells: claimed 15+ yrs vs. dates from 2016; two full-time jobs in two countries; a 6-day-old look-alike reference domain; a photo with no web footprint).
 - **Test 3 — Clean or compromised article?** a longer AI-drafted post with **three camouflaged injections** (a fake "reviewer note" that exfiltrates a list; a "system instruction" that redirects incident reports to an attacker; a mid-paragraph aside that hijacks the AI summary into phishing). Flag each compromised **section** and justify it.
- **Scoring:** T1 +120 (or +180 for correct "can't tell"); T2 +150 right-item-and-reason, +70 right item wrong reason, −80 false flag; T3 +220 / +90 / −90 similarly · timer 240s.
- **Lesson:** verify against independent evidence, name the specific tell, treat generated output as untrusted.

Round 4 — The review board (AI governance)

- **Concept:** three automated decisions flagged for human review; tag the single governance failure.
- **Mechanic:** pick **unfair bias** (hiring tool using postal code as a proxy), **no explanation** (unexplainable loan denial), or **privacy violation** (medical prioritisation using brokered browsing data). First answer locks.
- **Scoring:** +250 per correct diagnosis · timer 150s.

- **Lesson:** automated decisions fail in recognisable ways; naming the failure is the first step to challenging it.
-

8. UX / interaction principles applied throughout

- **Three-zone round layout** (incoming → proposed action → decision) keeps dense screens readable — the fix for "everything thrown at once".
 - **Objective + how-to on every round**, plus a persistent one-line objective bar.
 - **Reactive feedback:** breach/leak counters, red/green state flips, redaction blocks, animated timer bar.
 - **Plain language over jargon:** technical names are shown as muted tags beside plain-English labels so both HR and SDEs can reason.
 - **Fair-and-tense scoring** (independent per item) rather than compounding, so one early mistake doesn't doom a rankable run.
-

9. Constants reference (all tunable)

Constant	Value	Where	Meaning
GAME_TITLE / GAME_TAGLINE	"Tells" / ...	lib/config.ts	brand text
TIME_BONUS_PER_SEC	2	lib/config.ts	speed bonus rate
TIME_BONUS_CAP	300	lib/config.ts	max speed bonus
HINT_COST	75	lib/config.ts	cost per hint after the free one
TAB_PENALTY	150	lib/auth.tsx	points per tab switch
DQ_LIMIT	3	lib/auth.tsx	tab switches → disqualified
SEQUENTIAL_LOCK	false	lib/rounds.ts	enforce round order
CREDENTIALS	5 demo users / tells2026	lib/auth.tsx	login
R1 RIGHT/MISS/FALSE_POS	200 / 250 / 100	app/round-1	correct / miss-attack / false-block
R2 SOLVE/FIRST_TRY_BONUS/FAIL_COST	250 / 150 / 80	app/round-2	per system / first-try / wrong-open
R3 test points	120·180 / 150·70·80 / 220·90·90	app/round-3	T1 / T2 / T3
R4 per-correct	250	app/round-4	correct diagnosis
Round timers	240 / 240 / 240 / 150 s	each round	round length
Storage keys	chimera_game_v1 , tells_user_v1 , tells_anticheat_v1 , tells_leaderboard_v1	libs	localStorage

10. Rebuild from scratch

```
npx create-next-app@14 tells-app --ts --tailwind --app --eslint --src-dir=false --import-alias  
cd tells-app
```

Then, in order:

1. **Design tokens** — set the color palette in `tailwind.config.ts` (`bg/panel/panel2/line/line2/ink/ink2/ink3/acc/acc2/info/warn/danger/ok/crit`) and the base background + `::selection` + `.hidden-ink` rules in `app/globals.css`.
2. **Primitives** — `components/Icon.tsx` (one component, a `name` → `svg path` table).
3. **State** — `lib/types.ts` (`RoundResult`), `lib/gameState.tsx` (`GameProvider`), `lib/useCountdown.ts`, `lib/config.ts` (`title`, `timeBonus`, `HINT_COST`), `lib/rounds.ts` (`round registry` + `SEQUENTIAL_LOCK`).
4. **Shared chrome** — `HudBar`, `IntroOverlay`, `ObjectiveBar`, `DebriefOverlay`, `Hints`.
5. **Rounds** — one `app/round-N/page.tsx` at a time. Each: `intro` | `play` | `debrief` phase machine, its data arrays, refs-mirror-state scoring, `finalize()` that adds the time bonus and calls `recordResult`. Follow the three-zone layout.
6. **Mission map** (`app/page.tsx`) and **results** (`app/debrief/page.tsx`, two tabs).
7. **Auth & competition** — `lib/auth.tsx`, `components/Login.tsx`, `AuthGate`, `UserMenu`, `RulesModal`, `SessionGuard`, `lib/leaderboard.ts`; wire providers in `app/layout.tsx`.
8. **Atmosphere** — `MatrixRain`, `Fullscreen`, `Logo`, `app/icon.svg`; drop the real logo into `public/assets/`.

Verify each step with `npx tsc --noEmit` and `npx next build`.

11. How to extend / operate

- **Add a round:** add an entry to `ROUNDS` in `lib/rounds.ts`, create `app/round-N/page.tsx` following an existing round, and it appears on the map + report automatically.
- **Change credentials:** edit `CREDENTIALS` in `lib/auth.tsx` (or replace with a real auth call).
- **Swap the logo:** replace `public/assets/company-logo.png` (or update the `src` in `components/Logo.tsx`).
- **Enforce sequential play:** set `SEQUENTIAL_LOCK = true` in `lib/rounds.ts`.
- **Tune difficulty/scoring:** all values in §9 are single constants.
- **Rebrand:** `GAME_TITLE` / `GAME_TAGLINE` in `lib/config.ts`; palette in `tailwind.config.ts`.

12. Production hardening (do before a real, multi-player event)

Everything is client-side, which is perfect for design/demo and single-machine kiosks but **must change** for a live competition:

- **Move validation server-side.** All answer keys, the résumé headshot answer, the hidden-text payloads, and flag matching currently live in the client — inspectable in devtools. Serve rounds and grade submissions on a server.
- **Real auth + shared leaderboard.** Credentials and scores are in `localStorage`, per device. Use a real identity provider and a server-backed leaderboard so players compete across machines.
- **Server-side anti-cheat.** Tab-switch counting is best-effort in the browser; a determined player can bypass it. Corroborate server-side (submission timing, focus events posted to an API) if it must be

authoritative.

- **HELIX/LLM note:** there is no live model in the build — round logic is scripted pattern-matching, which is safe and deterministic for a CTF. If you ever want players injecting against a real LLM, that's a separate architecture.
-

13. Known limitations

- No persistence across devices (localStorage only).
- The Matrix rain pauses when the *browser tab* is backgrounded (normal rAF throttling).
- The `app/round-3` StyleGAN headshot is a generated face served statically; swap per brand/tone as needed.
- `lib/types.ts` still carries a couple of legacy types from an earlier phishing version of Round 1; harmless but prunable.